

CODE SPORT



In this month's column, we continue our discussion on deep learning.

While we have been discussing deep learning in the last few columns, a few readers asked me to share interview questions on this topic as well as on general machine learning. So, in this month's column, we focus on some of the common interview questions, and I request readers to send me your email replies to these questions. I will feature them in our next column when we discuss the answers to these questions.

1. In last month's column, we had discussed auto encoders and how they can be used for dimensionality reduction. I had mentioned that it is possible to achieve a dimensionality reduction of the input representation by having a hidden layer of smaller dimensions compared to the input representation dimensions. Now the question to our readers is the following: Is it always necessary to have a hidden layer of smaller dimensions in an auto-encoder? If you build an auto-encoder in which you have the hidden layer whose dimensions are larger than the input layer, can you justify why you chose to do so?
2. Continuing on the subject of auto-encoders, why do we really need them for dimensionality reduction? Most of you would be familiar with dimensionality reduction techniques such as Principal Components Analysis (PCA). What is the advantage of using auto-encoders for dimensionality reduction of data compared to a PCA based approach? (*Clue*: Think of linear vs non-linear.)
3. Given that deep neural networks need large amounts of training data to generalise well, can you explain methods by which you can artificially increase the size of your training data? (*Clue*: Think of transformations such as scaling, rotation and translation. While it may be obvious in the case of image data sets to employ these transformations to increase the size of training data, what kind of transformations can you employ in the case of text data?)
4. Regularisation techniques are widely used to avoid over-fitting of the neural network parameters to training data so that the constructed network can still generalise well for unseen test data. Can you explain the differences between L1 regularisation and L2 regularisation methods? When would you use the first option instead of the second? If you change the network learning rate, does it impact the choice of your regularisation parameter λ ?
5. You have been given a neural network and training data of size N . How would you determine whether the constructed network suffers from the problem of over-fitting?
6. One of the several other techniques used to avoid over-fitting is employing drop-out while training a neural network. In simple terms, drop-out implies having a percentage of neurons inactive while training the network for a fixed number of mini-epochs and then repeating this process a number of times, having a different set of neurons inactive each time. After completing training, the learnt weights of the neurons are scaled down. Can you explain why this is needed? Also, can you explain the intuition behind drop-out in terms of why it can help avoid over-fitting?
7. One way of thinking about the drop-out technique is that by having a certain percentage of neurons inactive for each learning phase, we actually train a different neural network and then combine these different networks to obtain the final network. Can you think of any other technique in machine learning whereby you can combine different weak learners to build a strong learner?
8. Different activation functions can be employed for the neurons such as rectilinear, tan-h, sigmoid, etc. Can you explain the factors involved in the choice of activation functions? Are there any criteria that a function needs to satisfy, for it to be considered as an activation